

Unit-5

Computer-Aided Software Engineering (Case)

Introduction:

we will discuss about computer aided software engineering (CASE) and how use of CASE tools help to improve software development effort and maintenance effort. Of late, CASE has emerged as a much talked topic in software industries. Software is becoming the costliest component in any computer installation. Even though hardware prices keep dropping like never and falling below even the most optimistic expectations, software prices are becoming costlier due to increased manpower costs. This scenario has got most managers worried. In this scene, CASE tools promise effort and cost reduction in software development and maintenance. Therefore, deployment and development of CASE tools have become pet subjects for most software project managers. For software engineers, CASE tools promises to take drudgery out of routine jobs, and help develop better quality products more efficiently.

With this brief introduction and motivation for studying CASE tools, we will first define the scope of CASE and examine the different concepts associated with CASE. Subsequently, we will discuss the features of different types of CASE tools.

CASE AND ITS SCOPE:

We first need to define what is a CASE tool and what is a CASE environment. A CASE tool is a generic term used to denote any form of automated support for software engineering. In a more restrictive sense a CASE tool can mean any tool used to automate some activity associated with software development. Many CASE tools are now available. Some of these tools assist in phase-related tasks such as specification, structured analysis, design, coding, testing, etc. and others to non-phase activities such as project management and configuration management. The primary objectives in using any CASE tool are:

- To increase productivity.
- To help produce better quality software at lower cost.

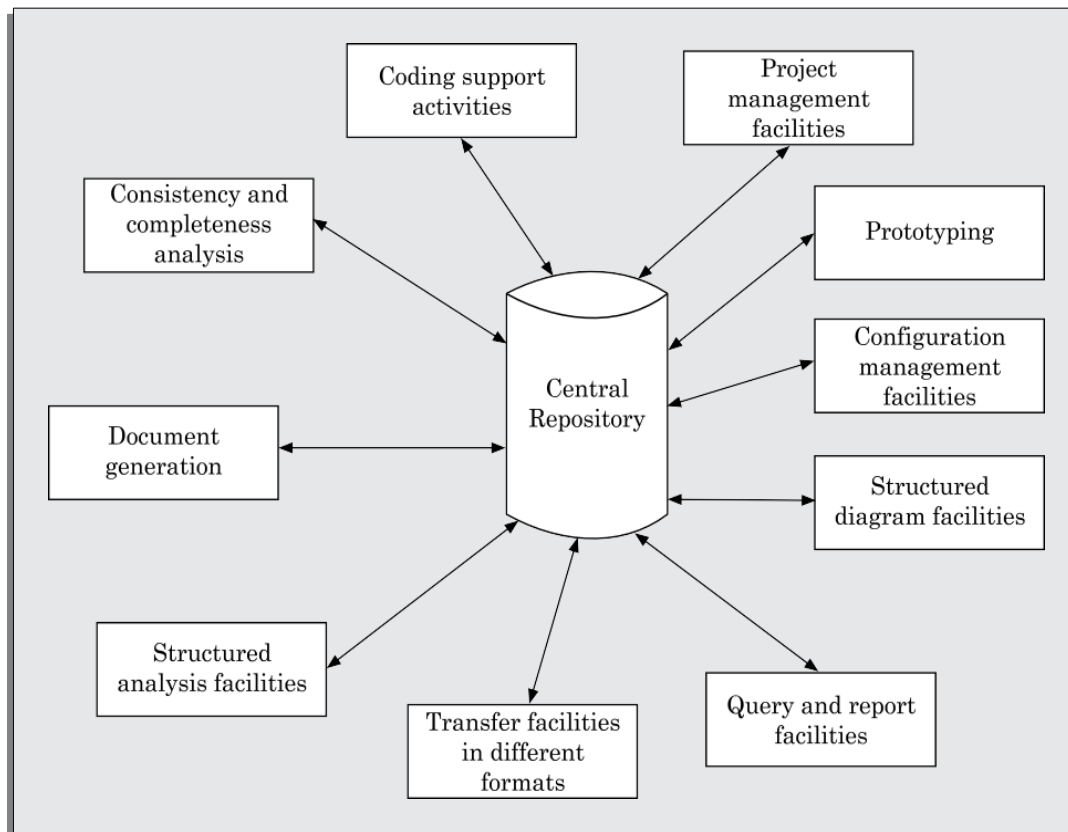
CASE ENVIRONMENT:

Although individual CASE tools are useful, the true power of a tool set can be realised only when these set of tools are integrated into a common framework or environment. If the different CASE tools are not integrated, then the data generated by one tool would have to input to the other tools. This may also involve format conversions as the tools developed by different vendors are likely to use different formats.

CASE tools are characterised by the stage or stages of software development life cycle on which they focus. Since different tools covering different stages share common information, it is required that they integrate through some central repository to have a consistent view of information associated with the software.

This central repository is usually a data dictionary containing the definition of all composite and elementary data items. Through the central repository all the CASE tools in a CASE environment share common information among themselves.

Thus a CASE environment facilitates the automation of the step-by-step methodologies for software development. In contrast to a CASE environment, a programming environment is an integrated collection of tools to support only the coding phase of software development. The tools commonly integrated in a programming environment are a text editor, a compiler, and a debugger.



A CASE environment.

The standard programming environments such as Turbo C, Visual C++, etc. come equipped with a program editor, compiler, debugger, linker, etc. All these tools are integrated. If you click on an error reported by the compiler, not only does it take you into the editor, but also takes the cursor to the specific line or statement causing the error.

Benefits of CASE:

Several benefits accrue from the use of a CASE environment or even isolated CASE tools. Let us examine some of these benefits:

- A key benefit arising out of the use of a CASE environment is cost saving through all developmental phases. Different studies carry out to measure the impact of CASE, put the effort reduction between 30 per cent and 40 per cent.
- Use of CASE tools leads to considerable improvements in quality. This is mainly due to the facts that one can effortlessly iterate through the different phases of software development, and the chances of human error is considerably reduced.
- CASE tools help produce high quality and consistent documents. Since the important data relating to a software product are maintained in a central repository, redundancy in the stored data is reduced, and therefore, chances of inconsistent documentation is reduced to a great extent.
- CASE tools take out most of the drudgery in a software engineer's work. For

example, they need not check meticulously the balancing of the DFDs, but can do it effortlessly through the press of a button.

- CASE tools have led to revolutionary cost saving in software maintenance efforts. This arises not only due to the tremendous value of a CASE environment in traceability and consistency checks, but also due to the systematic information capture during the various phases of software development as a result of adhering to a CASE environment.
- Introduction of a CASE environment has an impact on the style of working of a company, and makes it oriented towards the structured and orderly approach.

CASE SUPPORT IN SOFTWARE LIFE CYCLE:

Let us examine the various types of support that CASE provides during the different phases of a software life cycle. CASE tools should support a development methodology, help enforce the same, and provide certain amount of consistency checking between different phases. Some of the possible support that CASE tools usually provide in the software development life cycle are discussed below.

1. **Prototyping Support**
2. **Structured Analysis and Design**
3. **Code Generation**
4. **Test CASE Generator**

1. Prototyping Support:

We have already seen that prototyping is useful to understand the requirements of complex software products, to demonstrate a concept, to market new ideas, and so on. The prototyping CASE tool's requirements are as follows:

- Define user interaction.
- Define the system control flow.
- Store and retrieve data required by the system.
- Incorporate some processing logic.

A good prototyping tool should support the following features:

- Since one of the main uses of a prototyping CASE tool is *graphical user interface* (GUI) development, a prototyping CASE tool should support the user to create a GUI using a graphics editor. The user should be allowed to define all data entry forms, menus and controls.
- It should integrate with the data dictionary of a CASE environment.
- If possible, it should be able to integrate with external user defined modules written in C or some popular high level programming languages.
- The user should be able to define the sequence of states through which a created prototype can run. The user should also be allowed to control the running of the prototype.
- The run time system of prototype should support mock up run of the actual system and management of the input and output data.

2. Structured Analysis and Design:

- Several diagramming techniques are used for structured analysis and structured design.
- A CASE tool should support one or more of the structured analysis and design technique.
- The CASE tool should support effortlessly drawing analysis and design diagrams.
- The CASE tool should support drawing fairly complex diagrams and preferably

through a hierarchy of levels. It should provide easy navigation through different levels and through design and analysis.

- The tool must support completeness and consistency checking across the design and analysis and through all levels of analysis hierarchy.

3. Code Generation:

As far as code generation is concerned, the general expectation from a CASE tool is quite low. A reasonable requirement is traceability from source file to design data. More pragmatic support expected from a CASE tool during code generation phase are the following:

- The CASE tool should support generation of module skeletons or templates in one or more popular languages. It should be possible to include copyright message, brief description of the module, author name and the date of creation in some selectable format.
- The tool should generate records, structures, class definition automatically from the contents of the data dictionary in one or more popular programming languages.
- It should generate database tables for relational database management systems.
- The tool should generate code for user interface from prototype definition for X window and MS window based applications.

4. Test CASE Generator:

The CASE tool for test case generation should have the following features:

- It should support both design and requirement testing.
- It should generate test set reports in ASCII format which can be directly imported into the test plan document.

Other characteristics of CASE tools:

The characteristics listed in this section are not central to the functionality of CASE tools but significantly enhance the effectivity and usefulness of CASE tools.

Hardware and Environmental Requirements:

- In most cases, it is the existing hardware that would place constraints upon the CASE tool selection.
- Thus, instead of defining hardware requirements for a CASE tool, the task at hand becomes to fit in an optimal configuration of CASE tool in the existing hardware capabilities.
- The heterogeneous network is one instance of distributed environment and we choose this for illustration as it is more popular due to its machine independent features.
- The CASE tool implementation in heterogeneous network makes use of client-server paradigm.
- The tool set is integrated through the data dictionary which supports multiple projects, multiple users working simultaneously and allows to share information between users and projects.
- The tool should work satisfactorily for maximum possible number of users working simultaneously.

Documentation Support:

- The deliverable documents should be organized graphically and should be able to incorporate text and diagrams from the central repository.

- This helps in producing up-to- date documentation.
- The CASE tool should integrate with one or more of the commercially available desktop publishing packages.
- It should be possible to export text, graphics, tables, data dictionary reports to the DTP package in standard forms such as PostScript.

Project Management:

- It should support collecting, storing, and analysing information on the software project's progress such as the estimated task duration, scheduled and actual task start, completion date, dates and results of the reviews, etc.

External Interface:

- The tool should allow exchange of information for reusability of design.
- The information which is to be exported by the tool should be preferably in ASCII format and support open architecture.
- Similarly, the data dictionary should provide a programming interface to access information.
- It is required for integration of custom utilities, building new techniques, or populating the data dictionary.

Reverse Engineering Support:

- The tool should support generation of structure charts and data dictionaries from the existing source codes.
- It should populate the data dictionary from the source code. If the tool is used for re-engineering information systems, it should contain conversion tool from indexed sequential file structure, hierarchical and network database to relational database systems.

Data Dictionary Interface:

- The data dictionary interface should provide view and update access to the entities and relations stored in it.
- It should have print facility to obtain hard copy of the viewed screens.
- It should provide analysis reports like cross-referencing, impact analysis, etc.
- Ideally, it should support a query language to view its contents.

Tutorial and Help:

- The application of CASE tool and thereby its success depends on the users' capability to effectively use all the features supported.
- Therefore, for the uninitiated users, a tutorial is very important.
- The tutorial should not be limited to teaching the user interface part only, but should comprehensively cover the following points
- The tutorial should cover all techniques and facilities through logically classified sections.
- The tutorial should be supported by proper documentation.

TOWARDS SECOND GENERATION CASE TOOL:

An important feature of the second generation CASE tool is the direct support of any adapted methodology. This would necessitate the function of a CASE administrator for every organisation, who can tailor the CASE tool to a particular methodology. In addition, we may look forward to the following features in the second generation CASE tool:

Intelligent diagramming support: The fact that diagramming techniques are useful for system analysis and design is well established. The future CASE tools would provide help to aesthetically and automatically layout the diagrams.

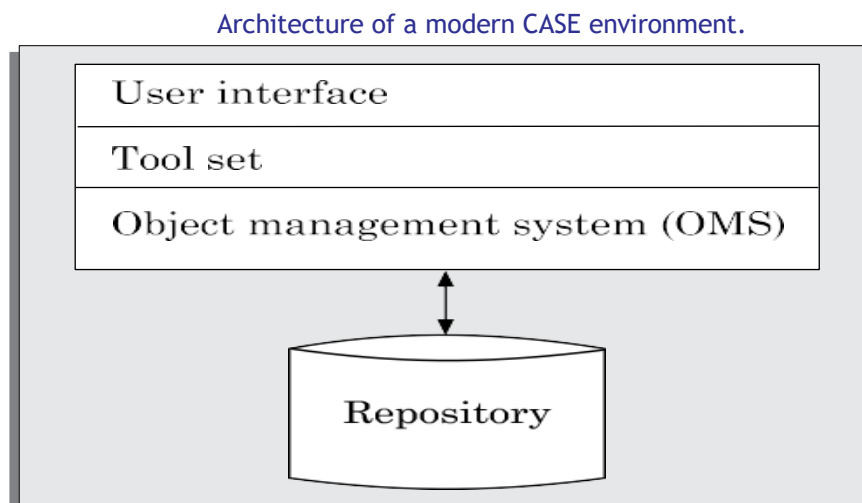
Integration with implementation environment: The CASE tools should provide integration between design and implementation.

Data dictionary standards: The user should be allowed to integrate many development tools into one environment. It is highly unlikely that any one vendor will be able to deliver a total solution. Moreover, a preferred tool would require tuning up for a particular system. Thus the user would act as a system integrator. This is possible only if some standard on data dictionary emerges.

Customisation support: The user should be allowed to define new types of objects and connections. This facility may be used to build some special methodologies. Ideally it should be possible to specify the rules of a methodology to a rule engine for carrying out the necessary consistency checks.

ARCHITECTURE OF A CASE ENVIRONMENT:

The architecture of a typical modern CASE environment is shown diagrammatically in bellow Figure . The important components of a modern CASE environment are user interface, tool set, object management system (OMS), and a repository. We have already seen the characteristics of the tool set. Let us examine the other components of a CASE environment.



User interface:

The user interface provides a consistent framework for accessing the different tools thus making it easier for the users to interact with the different tools and reducing the overhead of learning how the different tools are used.

Object management system and repository:

Different case tools represent the software product as a set of entities such as specification, design, text data, project plan, etc. The object management system maps these logical entities into the underlying storage management system (repository). The commercial relational database management systems are geared towards supporting large volumes of information structured as simple relatively short records. There are a few types of entities but large number of instances. By contrast, CASE tools create a large number of entities and relation types with perhaps a few instances of each. Thus the object management system takes care of appropriately mapping these entities into the underlying storage management system.

Software Maintenance:

Many students and practicing engineers have a preconceived bias against software maintenance work. The mention of the word maintenance brings up the image of a screwdriver, wielding mechanic with soiled hands holding onto a bagful of spare parts. It would be the objective of this chapter to clear up this misnomer, provide some intuitive understanding of the software maintenance projects, and to familiarise the reader with the latest techniques in software maintenance.

Software maintenance denotes any changes made to a software product after it has been delivered to the customer. Maintenance is inevitable for almost any kind of product. However, most products need maintenance due to the wear and tear caused by use. For example, a car tyre wears out due to use. On the other hand, software products do not need maintenance on this count, but need maintenance to correct errors, enhance features, port to new platforms, etc.

CHARACTERISTICS OF SOFTWARE MAINTENANCE:

Software maintenance is becoming an important activity of a large number of organisations. This is no surprise, given the rate of hardware obsolescence, the immortality of a software product per se, and the demand of the user community to see the existing software products run on newer platforms, run in newer environments, and/or with enhanced features. When the hardware platform changes, and a software product performs some low-level functions, maintenance is necessary.

Types of Software Maintenance:

There are three types of software maintenance, which are described as follows:

Corrective: Corrective maintenance of a software product is necessary to overcome the failures observed while the system is in use.

Adaptive: A software product might need maintenance when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware or software.

Perfective: A software product needs maintenance to support any new features that the users may want it to support, to change different functionalities of the system according to customer demands, or to enhance the performance of the system.

Characteristics of Software Evolution:

Lehman and Belady studied the characteristics of evolution of several software products [1980]. They expressed their observations in the form of laws. Their important laws are presented in the following subsection. But a word of caution here is that these are generalisations and may not be applicable to specific cases. Further, most of their observations concern large software projects and may not be appropriate for the maintenance and evolution of very small products.

Lehman's first law: A software product must change continually or become progressively less useful. Every software product continues to evolve after its development through maintenance efforts. Larger products stay in operation for longer times because of higher replacement costs and therefore tend to incur higher maintenance efforts.

Lehman's second law: The structure of a program tends to degrade as more and more maintenance is carried out on it. The reason for the degraded structure is that usually maintenance activities result in patch work. It is rarely the case that members of the original development team are part of the maintenance team.

Lehman's third law: Over a program's lifetime, its rate of development is approximately constant. The rate of development can be quantified in terms of the lines of code written or modified. Therefore this law states that the rate at which code is written or modified is approximately the same during development and maintenance.

Special Problems Associated with Software Maintenance:

Software maintenance work currently is typically much more expensive than what it should be and takes more time than required. The reasons for this situation are the following:

Software maintenance work in organisations is mostly carried out using ad hoc techniques. The primary reason being that software maintenance is one of the most neglected areas of software engineering.

Software maintenance has a very poor image in industry. Therefore, an organisation often cannot employ bright engineers to carry out maintenance work. Even though maintenance suffers from a poor image, the work involved is often more challenging than development work. During maintenance it is necessary to thoroughly understand someone else's work, and then carry out the required modifications and extensions.

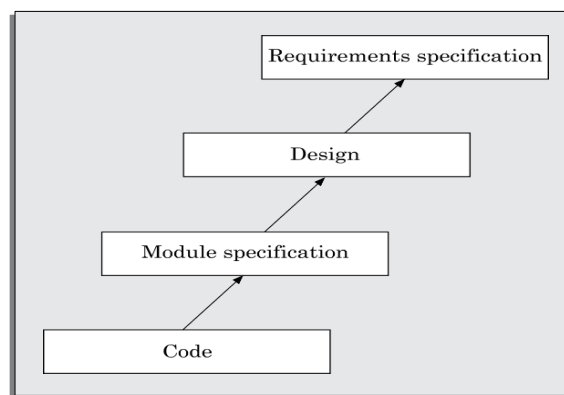
Another problem associated with maintenance work is that the majority of software products needing maintenance are legacy products. Though the word legacy implies "aged" software, but there is no agreement on what exactly is a legacy system. It is prudent to define a legacy system as any software system that is hard to maintain.

SOFTWARE REVERSE ENGINEERING:

Software reverse engineering is the process of recovering the design and the requirements specification of a product from an analysis of its code. The purpose of reverse engineering is to facilitate maintenance work by improving the understandability of a system and to produce the necessary documents for a legacy system. Reverse engineering is becoming important, since legacy software products lack proper documentation, and are highly

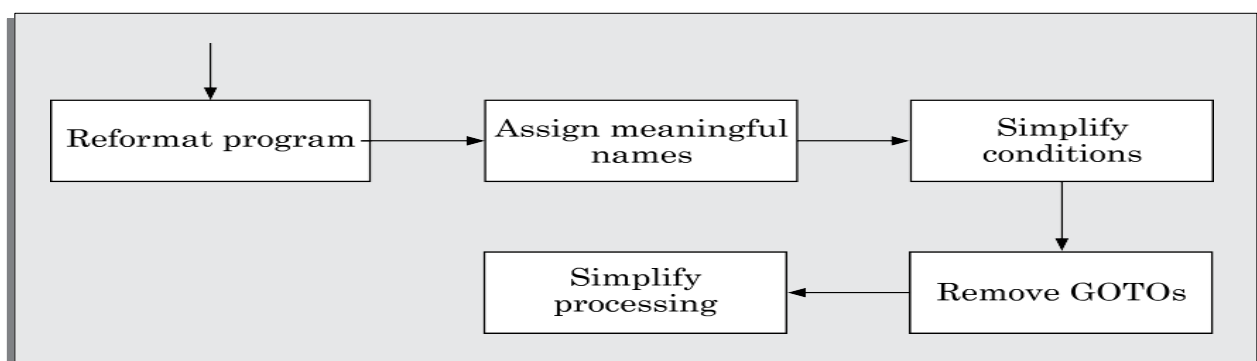
unstructured. Even well-designed products become legacy software as their structure degrades through a series of maintenance efforts.

The first stage of reverse engineering usually focuses on carrying out cosmetic changes to the code to improve its readability, structure, and understandability, without changing any of its functionalities. A way to carry out these cosmetic changes is shown schematically in bellow Figur. A program can be reformatted using any of the several available PrettyPrinter programs which layout the program neatly. Many legacy software products are difficult to comprehend with complex control structure and unthoughtful variable names. Assigning meaningful variable names is important because we had seen in Chapter 9 that meaningful variable names is the most helpful code documentation. All variables, data structures, and functions should be assigned meaningful names wherever possible. Complex nested conditionals in the program can be replaced by simpler conditional statements or whenever appropriate by case statements.



A process model for reverse engineering.

After the cosmetic changes have been carried out on a legacy software, the process of extracting the code, design, and the requirements specification can begin. These activities are schematically shown in bellow Figure . In order to extract the design, a full understanding of the code is needed. Some automatic tools can be used to derive the data flow and control flow diagram from the code. The structure chart (module invocation sequence and data interchange among modules) should also be extracted. The SRS document can be written once the full code has been thoroughly understood and the design extracted.



Cosmetic changes carried out before reverse engineering.

SOFTWARE MAINTENANCE PROCESS MODELS:

Before discussing process models for software maintenance, we need to analyse various activities involved in a typical software maintenance project. The activities involved in a software maintenance project are not unique and depend on several factors such as:

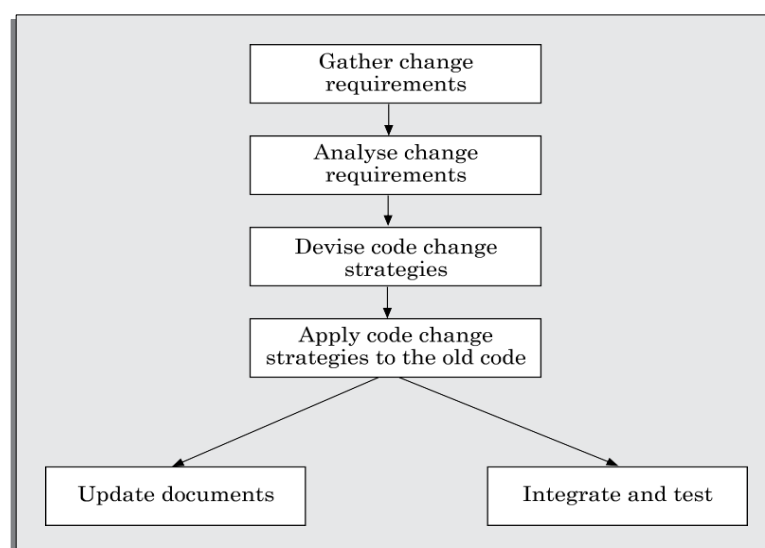
- (i) the extent of modification to the product required,
- (ii) the resources available to the maintenance team,
- (iii) the conditions of the existing product (e.g., how structured it is, how well documented it is, etc.), (iii) the expected project risks, etc.

When the changes needed to a software product are minor and straightforward, the code can be directly modified and the changes appropriately reflected in all the documents.

Since the scope (activities required) for different maintenance projects vary widely, no single maintenance process model can be developed to suit every kind of maintenance project. However, two broad categories of process models can be proposed.

First model:

The first model is preferred for projects involving small reworks where the code is changed directly and the changes are reflected in the relevant documents later. This maintenance process is graphically presented in bellow Figure. In this approach, the project starts by gathering the requirements for changes. The requirements are next analysed to formulate the strategies to be adopted for code change. At this stage, the association of at least a few members of the original development team goes a long way in reducing the cycle time, especially for projects involving unstructured and inadequately documented code. The availability of a working old system to the maintenance engineers at the maintenance site greatly facilitates the task of the maintenance team as they get a good insight into the working of the old system and also can compare the working of their modified system with the old system. Also, debugging of the re-engineered system becomes easier as the program traces of both the systems can be compared to localise the bugs.

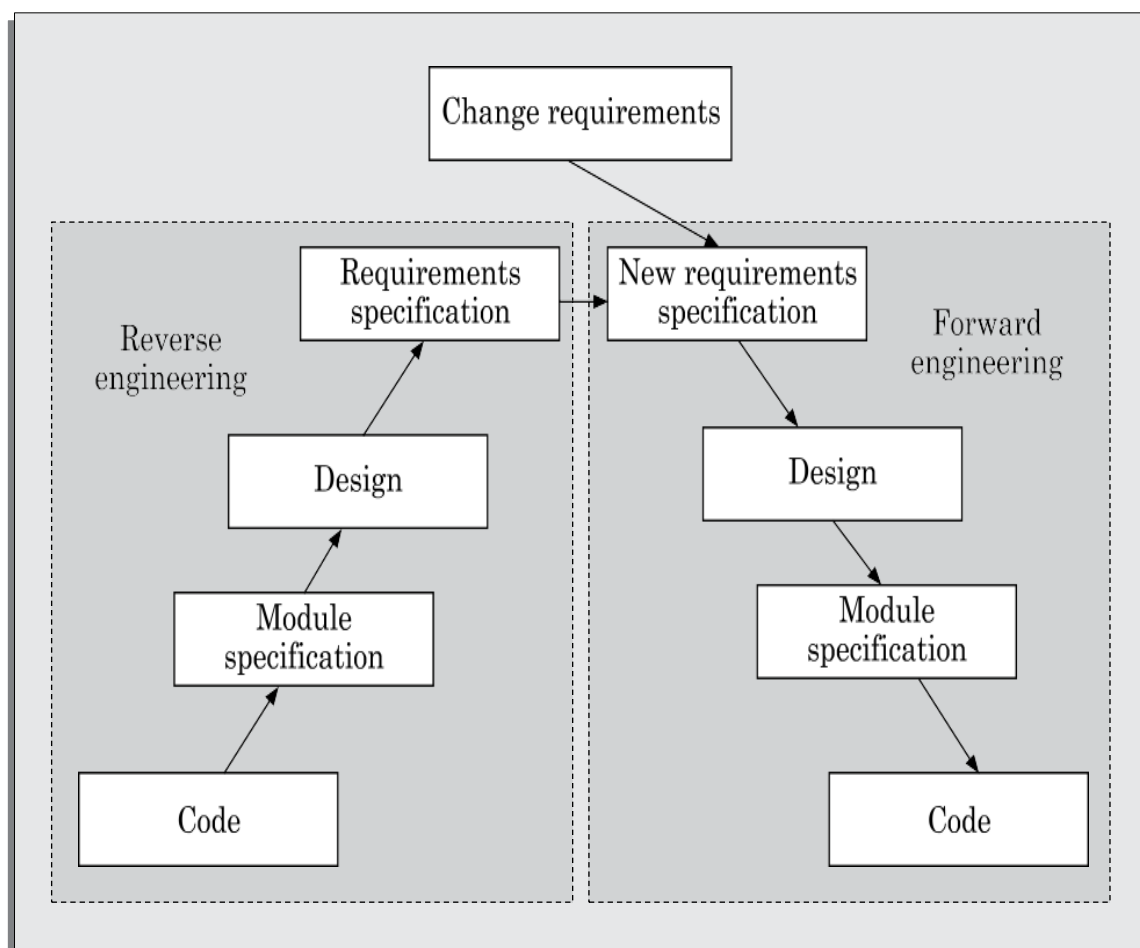


Maintenance process model 1.

Second model:

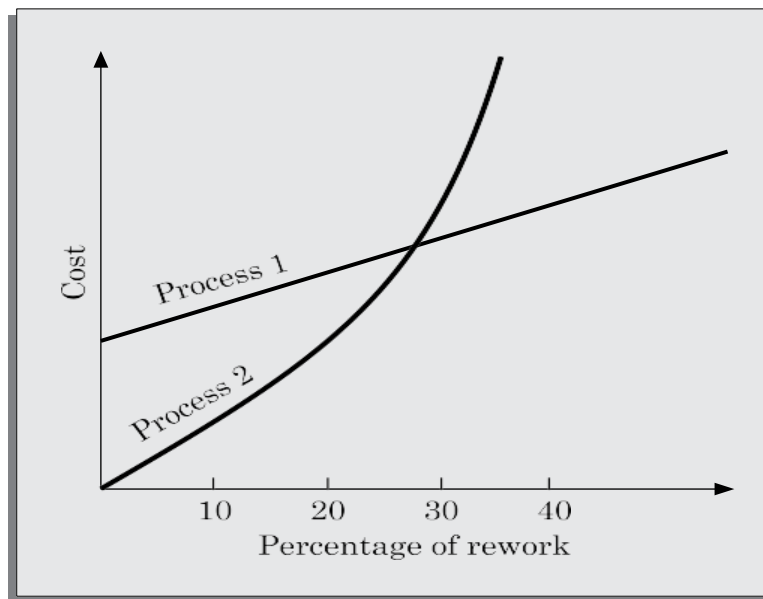
The second model is preferred for projects where the amount of rework required is significant. This approach can be represented by a reverse engineering cycle followed by a forward engineering cycle. Such an approach is also known as software re-engineering. This process model is depicted in bellow Figure.

The reverse engineering cycle is required for legacy products. During the reverse engineering, the old code is analysed (abstracted) to extract the module specifications. The module specifications are then analysed to produce the design. The design is analysed (abstracted) to produce the original requirements specification. The change requests are then applied to this requirements specification to arrive at the new requirements specification. At this point a forward engineering is carried out to produce the new code. At the design, module specification, and coding a substantial reuse is made from the reverse engineered products. An important advantage of this approach is that it produces a more structured design compared to what the original product had, produces good documentation, and very often results in increased efficiency.



Maintenance process model 2.

The efficiency improvements are brought about by a more efficient design. However, this approach is more costly than the first approach. An empirical study indicates that process 1 is preferable when the amount of rework is no more than 15 per cent (see bellow Figure).



Empirical estimation of maintenance cost versus percentage rework.

Besides the amount of rework, several other factors might affect the decision regarding using process model 1 over process model 2 as follows:

- Re-engineering might be preferable for products which exhibit a high failure rate.
- Re-engineering might also be preferable for legacy products having poor design and code structure.

ESTIMATION OF MAINTENANCE COST:

We had earlier pointed out that maintenance efforts require about 60 per cent of the total life cycle cost for a typical software product. However, maintenance costs vary widely from one application domain to another. For embedded systems, the maintenance cost can be as much as 2 to 4 times the development cost.

Boehm [1981] proposed a formula for estimating maintenance costs as part of his COCOMO cost estimation model. Boehm's maintenance cost estimation is made in terms of a quantity called the annual change traffic (ACT). Boehm defined ACT as the fraction of a software product's source instructions which undergo change during a typical year either through addition or deletion.

$$ACT = \frac{KLOC_{added} + KLOC_{deleted}}{KLOC_{total}}$$

where, $KLOC_{added}$ is the total kilo lines of source code added during maintenance. $KLOC_{deleted}$ is the total KLOC deleted during maintenance. Thus, the code that is changed, should be counted in both the code added and code deleted.

The annual change traffic (ACT) is multiplied with the total development cost to arrive at the maintenance cost:

$$\text{Maintenance cost} = ACT \times \text{Development cost}$$

Most maintenance cost estimation models, however, give only approximate results because they do not take into account several factors such as experience level of the engineers, and familiarity of the engineers with the product, hardware requirements, software complexity, etc.

Software Reuse:

Introduction:

Software products are expensive. Therefore, software project managers are always worried about the high cost of software development and are desperately looking for ways to cut development cost. A possible way to reduce development cost is to reuse parts from previously developed software. In addition to reduced development cost and time, reuse also leads to higher quality of the developed products since the reusable components are ensured to have high quality. A reuse approach that is of late gaining prominence is component- based development. Component-based software development is different from the traditional software development in the sense that software is developed by assembling software from off-the-shelf components.

Software development with reuse is very similar to a modern hardware engineer building an electronic circuit by using standard types of ICs and other components. In this Chapter, we will review the state of art in software reuse.

WHAT CAN BE REUSED:

Definition:

Before discussing the details of reuse techniques, it is important to deliberate about the kinds of the artifacts associated with software development that can be reused. Almost all artifacts associated with software development, including project plan and test plan can be reused. However, the prominent items that can be effectively reused are:

- Requirements specification
- Design
- Code
- Test cases
- Knowledge

Knowledge is the most abstract development artifact that can be reused. Out of all the reuse artifacts, reuse of knowledge occurs automatically without any conscious effort in this direction. However, two major difficulties with unplanned reuse of knowledge is that a developer experienced in one type of product might be included in a team developing a different type of software. Also, it is difficult to remember the details of the potentially reusable development knowledge. A planned reuse of knowledge can increase the effectiveness of reuse. For this, the reusable knowledge should be systematically extracted and documented. But, it is usually very difficult to extract and document reusable knowledge.

WHY ALMOST NO REUSE SO FAR:

A common scenario in many software development industries is explained further. Engineers working in software development organisations often have a feeling that the current system that they are developing is similar to the last few systems built. However, no attention is paid on how not to duplicate what can be reused from previously developed systems. Everything is being built from scratch. The current system falls behind schedule and no one has time to figure out how the similarity between the current system and the systems developed in the past can be exploited.

Even those organisations which embark on a reuse program, in spite of the above difficulty, face other problems. Creation of components that are reusable in different applications is a difficult problem. It is very difficult to anticipate the exact components that can be reused across different applications. But, even when the reusable components are carefully created and made available for reuse, programmers prefer to create their own, because the available components are difficult to understand and adapt to the new applications.

In this context, the following observation is significant: The routines of mathematical libraries are being reused very successfully by almost every programmer. No one in their mind would think of writing a routine to compute sine or cosine. Let us investigate why reuse of commonly used mathematical functions is so easy. Several interesting aspects emerge. Cosine means the same to all. Everyone has clear ideas about what kind of argument should cosine take, the type of processing to be carried out and the results returned. Secondly, mathematical libraries have a small interface. For example, cosine requires only one parameter. Also, the data formats of the parameters are standardised. These are some fundamental issues which would remain valid for all our subsequent discussions on reuse. In the following section, we discuss the issues that must be addressed while starting any reuse program in an organisation.

BASIC ISSUES IN ANY REUSE PROGRAM:

The following are some of the basic issues that must be clearly understood for starting any reuse program:

- **Component creation**
- **Component indexing and storing**
- **Component search**
- **Component understanding**
- **Component adaptation**
- **Repository maintenance**

Component creation: For component creation, the reusable components have to be first identified. Selection of the right kind of components having potential for reuse is important. In Section 14.4, we discuss domain analysis as a promising technique which can be used to create reusable components.

Component indexing and storing: Indexing requires classification of the reusable components so that they can be easily searched when we look for a component for reuse. The components need to be stored in a relational database management system (RDBMS) or an object-oriented database system (ODBMS) for efficient access when the number of components becomes large.

Component searching: The programmers need to search for right components matching their requirements in a database of components. To be able to search components efficiently, the programmers require a proper method to describe the components that they are looking for.

Component understanding: The programmers need a precise and sufficiently complete understanding of what the component does to be able to decide whether they can reuse the component. To facilitate understanding, the components should be well documented and should do something simple.

Component adaptation: Often, the components may need adaptation before they can be reused, since a selected component may not exactly fit the problem at hand. However, tinkering with the code is also not a satisfactory solution because this is very likely to be a source of bugs.

Repository maintenance: A component repository once is created requires continuous maintenance. New components, as and when created have to be entered into the repository. The faulty components have to be tracked. Further, when new applications emerge, the older applications become obsolete. In this case, the obsolete components might have to be removed from the repository.

A REUSE APPROACH:

A promising approach that is being adopted by many organisations is to introduce a building block approach into the software development process. For this, the reusable components need to be identified after every development project is completed. The reusability of the identified components has to be enhanced and these have to be cataloged into a component library. It must be clearly understood that an issue crucial to every reuse effort is the identification of reusable components. Domain analysis is a promising approach to identify reusable components.

Domain Analysis: The aim of domain analysis is to identify the reusable components for a problem domain.

Reuse domain:

A reuse domain is a technically related set of application areas. A body of information is considered to be a problem domain for reuse, if a deep and comprehensive relationship exists among the information items as characterised by patterns of similarity among the development components of the software product. A reuse domain is a shared that show some coherence. Examples of domains are accounting software domain, banking software domain, business software domain, manufacturing automation software domain, telecommunication software domain, etc.

Evolution of a reuse domain:

The ultimate results of domain analysis is development of problem-oriented languages. The problem-oriented languages are also known as application generators. These application generators, once developed form application development standards. The domains slowly develop. As a domain develops, we may distinguish the various stages it undergoes:

Stage 1: There is no clear and consistent set of notations. Obviously, no reusable components are available. All software is written from scratch.

Stage 2: Here, only experience from similar projects are used in a development effort. This means that there is only knowledge reuse.

Stage 3: At this stage, the domain is ripe for reuse. The set of concepts are stabilised and the notations standardised. Standard solutions to standard problems are available. There is both knowledge and component reuse.

Stage 4: The domain has been fully explored. The software development for the domain can largely be automated. Programs are not written in the traditional sense any more. Programs are written using a domain specific language, which is also known as an application generator.

Component Classification:

Components need to be properly classified in order to develop an effective indexing and storage scheme. We have already remarked that hardware reuse has been very successful. If we look at the classification of hardware components for clue, then we can observe that hardware components are classified using a multilevel hierarchy. At the lowest level, the components are described in several forms—natural language description, logic schema, timing information, etc. The higher the level at which a component is described, the more is the ambiguity. This has motivated the Prieto-Diaz's classification scheme.

Prieto-Diaz's classification scheme:

Each component is best described using a number of different characteristics or facets. For example, objects can be classified using the following:

- Actions they embody
- Objects they manipulate
- Data structures used
- Systems they are part of, etc.

Prieto-Diaz's faceted classification scheme requires choosing an n-tuple that best fits a component. Faceted classification has advantages over enumerative classification. Strictly enumerative schemes use a pre-defined hierarchy. Therefore, these force you to search for an item that best fits the component to be classified. This makes it very difficult to search a required component. Though cross referencing to other items can be included, the resulting network becomes complicated.

Searching:

The domain repository may contain thousands of reuse items. In such large domains, what is the most efficient way to search an item that one is looking for? A popular search technique that has proved to be very effective is one that provides a web interface to the repository. Using such a web interface, one would search an item using an approximate automated search using key words, and then from these results would do a browsing using the links provided to look up related items. The approximate automated search locates products that appear to fulfill some of the specified requirements. The items located through the approximate search serve as a starting point for browsing the repository.

Repository Maintenance:

Repository maintenance involves entering new items, retiring those items which are no more necessary, and modifying the search attributes of items to improve the effectiveness of search. Also, the links relating the different items may need to be modified to improve the effectiveness of search. The software industry is always trying to implement something that has not been quite done before. As patterns requirements emerge, new reusable components are identified, which may ultimately become more or less the standards. However, as technology advances, some components which are still reusable, do not fully address the current requirements. On the other hand, restricting reuse to highly mature components, can sacrifice potential reuse opportunity.

Reuse without Modifications:

Once standard solutions emerge, no modifications to the program parts may be necessary. One can directly plug in the parts to develop his application. Reuse without modification is much more useful than the classical program libraries. These can be supported by compilers through linkage to run-time support routines (application generators).

Application generators translate specifications into application programs. The specification usually is written using 4GL. The specification might also be in a visual form. The programmer would create a graphical drawing using some standard available symbols.

Application generators have significant advantages over simple parameterised programs. The biggest of these is that the application generators can express the variant information in an appropriate language rather than being restricted to function parameters, named constants, or tables.

Application generators have been applied successfully to data processing application, user interface, and compiler development. Application generators are less successful with the development of applications with close interaction with hardware such as real-time systems.

REUSE AT ORGANISATION LEVEL:

Reusability should be a standard part in all software development activities including specification, design, implementation, test, etc. Ideally, there should be a steady flow of reusable components. In practice, however, things are not so simple.

Extracting reusable components from projects that were completed in the past presents an important difficulty not encountered while extracting a reusable component from an ongoing project—typically, the original developers are no longer available for consultation. Development of new systems leads to an assortment of products, since reusability ranges from items whose reusability is immediate to those items whose reusability is highly improbable.

Achieving organisation-level reuse requires adoption of the following steps:

- Assess of an item's potential for reuse
- Refine the item for greater reusability
- Enter the product in the reuse repository

Assessing a product's potential for reuse:

Assessment of a components reuse potential can be obtained from an analysis of a questionnaire circulated among the developers. The questionnaire can be devised to assess a component's reusability. The programmers working in similar application domain can be used to answer the questionnaire about the product's reusability.

A sample questionnaire to assess a component's reusability is the following:

- Is the component's functionality required for implementation of systems in the future?
- How common is the component's function within its domain?
- Would there be a duplication of functions within the domain if the component is taken up?
- Is the component hardware dependent?
- Is the design of the component optimised enough?

- If the component is non-reusable, then can it be decomposed to yield some reusable components?
- Can we parametrise a non-reusable component so that it becomes reusable?

Refining products for greater reusability:

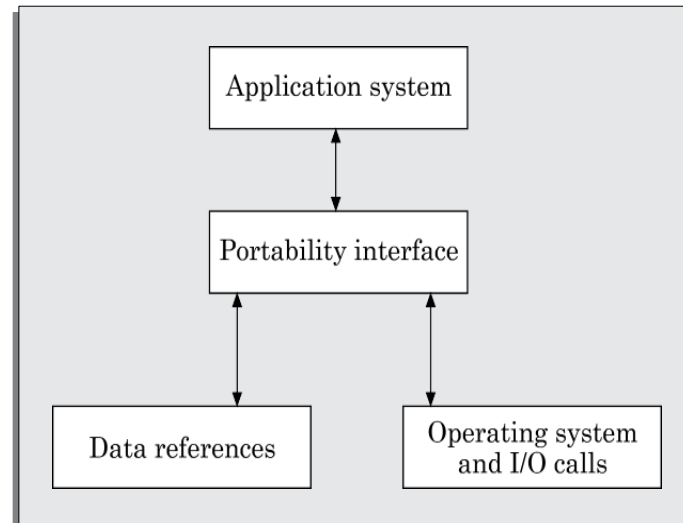
For a product to be reusable, it must be relatively easy to adapt it to different contexts. Machine dependency must be abstracted out or localised using data encapsulation techniques. The following refinements may be carried out.

Name generalisation: The names should be general, rather than being directly related to a specific application.

Operation generalisation: Operations should be added to make the component more general. Also, operations that are too specific to an application can be removed.

Exception generalisation: This involves checking each component to see which exceptions it might generate. For a general component, several types of exceptions might have to be handled.

Handling portability problems: Programs typically make some assumption regarding the representation of information in the underlying machine. These assumptions are in general not true for all machines. The programs also often need to call some operating system functionality and these calls may not be the same on all machines. Also, programs use some function libraries, which may not be available on all host machines. A portability solution to overcome these problems is shown in below Figure. The portability solution suggests that rather than call the operating system and I/O procedures directly, abstract versions of these should be called by the application program.



Improving reusability of a component by using a portability interface.

Current State of Reuse:

In spite of all the shortcomings of the state-of-the-art reuse techniques, it is the experience of several organisations that most of the factors inhibiting an effective reuse program are non-technical. Some of these factors are the following:

- Need for commitment from the top management.
- Adequate documentation to support reuse.
- Adequate incentive to reward those who reuse. Both the people contributing new

reusable components and those reusing the existing components should be rewarded to start a reuse program and keep it going.

- Providing access to and information about reusable components. Organisations are often hesitant to provide an open access to the reuse repository for the fear of the reuse components finding a way to their competitors.